Documento de Contexto Técnico y Funcional

Proyecto: Gestión Empresarial (Sistema Web Multiempresa ERP Textil) | Fecha: 22 de octubre de 2026

1. Resumen Ejecutivo y Propósito

1.1 Naturaleza del Proyecto

Gestión Empresarial es una plataforma web monolítica modular y multiempresa (multi-tenant) diseñada específicamente para la gestión operativa, administrativa, contable y de recursos humanos de empresas del sector textil, importación y distribución comercial. Soporta tres entidades comerciales operativas:

Código	Empresa	Razón Social	País	Moneda	Impuesto
`VICMAR_HN`	Vicmar	Telas Americanas Vicmar, S. de R.L.	Honduras (HN)	HNL (L)	RTN (15%)
`TELAS_OLIMPICA_MX`	Telas Olímpica	Telas Olímpica, S.A. de C.V.	México (MX)	MXN ($)	RFC (16%)
`GRUPO_CIRSA_HN`	Grupo Cirsa	Grupo Cirsa, S. de R.L.	Honduras (HN)	HNL (L)	RTN (15%)

1.2 Problema que Resuelve

Centraliza en una única infraestructura la dispersión operativa de compras de telas, inventarios por lotes/rollos, pedidos comerciales, seguimiento de ventas, control de expedientes de personal y estados financieros contables bajo reglas fiscales y monetarias independientes por país, eliminando el uso de hojas de cálculo aisladas y garantizando trazabilidad con números de serie automáticos por transacción.

1.3 Actores del Sistema (RBAC)

Rol	Módulos Permitidos	Responsabilidades Clave
Administrador	`compras`, `ventas`, `inventarios`, `contabilidad`, `costos`, `reportes` + admin	Acceso total, aprobación/rechazo de registros de usuarios, gestión de empleados y catálogos.
Compras	`compras`	Proveedores, requisiciones, órdenes de compra, recepciones de mercancía, devoluciones y cuentas por pagar.
Ventas	`ventas`	Clientes, cotizaciones, pedidos comerciales, facturación y cuentas por cobrar.
Inventarios	`inventarios`	Stock de rollos/metros, movimientos de almacén, ajustes y trazabilidad por lote.
Contabilidad	`contabilidad`, `costos`	Balances de comprobación, estados de resultados, balance general, costeo y nómina.
Empleado	Ninguno (solo perfil personal)	Acceso restringido a consulta de perfil propio y datos básicos.

2. Stack Tecnológico y Entorno

Capa	Tecnología / Librería	Versión / Detalle Técnico
Runtime Backend	Node.js	`>= 18.x` (Verificado en v24.20.0)
Servidor HTTP	Módulo nativo `http`	`http.createServer` sin Express; enrutamiento y rate-limit nativos en memoria
Driver Base de Datos	`pg` (node-postgres)	`^8.16.3` con `pg.Pool` y soporte TLS/SSL
DB Fallback / Testing	`pg-mem`	`^3.0.14` (Base de datos PostgreSQL en memoria cuando no se define URI externa)
Motor Base de Datos	Supabase PostgreSQL	PostgreSQL 15+ en AWS (Pooler puerto 5432/6543)
Criptografía	Node.js `crypto` nativo	PBKDF2 (100,000 iteraciones, sha512) y tokens de sesión SHA-256
Frontend	Vanilla JS (ES6+), HTML5, CSS3	Zero-Build (sin Webpack/Vite), diseño responsivo corporativo
Test Runner	Node.js Test Runner	`node --test test/*.test.js`

3. Arquitectura y Estructura de Directorios

3.1 Patrón Arquitectónico

El sistema aplica una **Arquitectura en Capas Limpia (Layered Monolith)** con separación estricta de responsabilidades:

- **Capa de Presentación (`SRC/Frontend`):** Vistas HTML semánticas servidas estáticamente por el backend que consumen la API REST mediante `fetch`.
- **Capa de Control y Enrutamiento (`SRC/Backend/server.js`):** Despacho de rutas HTTP, control CORS, cabeceras de seguridad (`X-Content-Type-Options: nosniff`, `Cache-Control: no-store`) y limitador de tasa por IP.
- **Capa de Dominio / Módulos (`SRC/Backend/modules/`):** Reglas de negocio de empleados, contabilidad, validación de permisos y asignación transaccional de números de serie.
- **Capa de Persistencia (`SRC/Backend/config/`):** Pool de conexiones PostgreSQL, traductor de placeholders SQL (`? -> $1..$n`), manejo de transacciones ACID y migraciones defensivas automáticas al arranque (`ensure*Tables`).

3.2 Árbol de Directorios

```
Gestion-Empresarial/
├── README.md                          # Guía de instalación y conexión a Supabase
└── SRC/
    ├── Backend/
    │   ├── .env.example               # Plantilla de variables de entorno
    │   ├── package.json               # Dependencias (pg, pg-mem) y scripts
    │   ├── server.js                  # Servidor HTTP, router API y servidor de estáticos
    │   ├── config/
    │   │   ├── connection.js          # Pool PostgreSQL, adaptador pg-mem y helper SQL
    │   │   ├── connection-errors.js   # Diagnóstico y sanitización de errores de red
    │   │   ├── db.js                  # Autenticación, sesiones, usuarios y empresas
    │   │   ├── mailer.js              # Notificaciones de recuperación de contraseña
    │   │   └── permissions.js         # Matriz RBAC de módulos por rol
    │   ├── database/
    │   │   └── schema.sql             # DDL idempotente para Supabase PostgreSQL
    │   ├── modules/
    │   │   ├── accounting.js          # Motor de reportes financieros y nómina
    │   │   ├── employees.js           # CRUD de empleados y validaciones por país
    │   │   └── records.js             # Datasets JSON operativos y secuencias de serie
    │   └── test/
    │       └── connection-errors.test.js # Suite de pruebas unitarias
    └── Frontend/
        ├── index.html, login.html, registrar-ingreso.html, recuperar.html, restablecer.html
        ├── dashboard.html, modulo.html, inventario.html, catalogos.html
        ├── empleados.html, empleado.html, admin.html, permisos.html
        ├── reportes-contables.html, cuenta.html, configuracion.html, bitacora.html
        └── assets/
            ├── css/styles.css         # Hoja de estilos corporativa unificada
            ├── icons/*.svg            # Iconografía vectorial del sistema
            └── js/                    # Lógica cliente (auth, session, module, admin, etc.)
```

4. Modelos de Datos y Entidades Principales

Tabla PostgreSQL	Clave Primaria / Única	Atributos Principales y Propósito
`companies`	PK: `id` | UQ: `code`, `name`	`code`, `name`, `legal_name`, `rtn`, `country_code`, `currency_code`, `tax_id_label`, `tax_rate`, `city`, `active`.
`users`	PK: `id` | UQ: `email`, `username`	`first_name`, `last_name`, `email`, `username`, `password_hash`, `company_id` (FK), `role`, `is_global_admin`, `status`.
`registration_requests`	PK: `id`	Solicitudes de alta pendientes de aprobación administrativa (`Pendiente`, `Aprobada`, `Rechazada`).
`auth_sessions`	PK: `id` | UQ: `token_hash`	`user_id`, `token_hash` (CHAR 64 SHA-256), `expires_at`, `revoked_at`.
`password_recovery_tokens`	PK: `id` | UQ: `token`	`user_id`, `token`, `expires_at`, `used_at`.
`business_datasets`	PK: `id` | UQ: `(company_id, module_key, view_key)`	`rows_json` (TEXT con matriz JSON de filas operativas), `updated_by`, `updated_at`.
`business_serial_sequences`	PK: `(company_id, module_key, view_key)`	`next_value` (BIGINT). Secuenciador transaccional con bloqueo `FOR UPDATE`.
`employees`	PK: `id` | UQ: `(company_id, email)`	`identity_number`, `tax_number`, `social_security_number`, `position`, `branch`, `system_role`, `status`, `deleted_at`.
`accounting_balances`	PK: `id` | UQ: `(company_id, period, account_code)`	`account_name`, `account_type`, `opening_balance`, `debit`, `credit`.
`payroll_concepts`	PK: `id` | UQ: `(company_id, concept_key)`	`description`, `status`, `integrates_salary`, `calculation_scope`, `benefit`, `movement_type`, `accumulated`.
`sales_profit_lines`	PK: `id` | UQ: `(company_id, period, product_code)`	`description`, `quantity`, `total_cost`, `total_price`.
`purchase_invoices` / `_lines`	PK: `id` | UQ: `(company_id, invoice_number)`	Cabecera y detalle de facturas de compra e importación textil.

5. Módulos y Flujos Principales (Lógica de Negocio)

5.1 Flujo de Autenticación y Sesiones (`config/db.js`)

- **Solicitud de Registro (`POST /api/auth/register`):** Valida formato estricto de correo electrónico (con detección de errores de tipeo como `gmial.com`), complejidad de clave y registra la petición en estado `Pendiente`.
- **Aprobación Administrativa (`POST /api/admin/registrations/approve`):** Un administrador aprueba la solicitud y el sistema traslada la cuenta a `users` conservando el hash PBKDF2.
- **Inicio de Sesión (`POST /api/auth/login`):** Verifica credenciales mediante `crypto.timingSafeEqual`, emite un token aleatorio de 32 bytes y guarda su hash SHA-256 en `auth_sessions` por 12 horas.

- **Bootstrap Admin:** En desarrollo, si no hay un administrador global activo, crea automáticamente el usuario `administrador` (`administrador@vicmar.hn`).

5.2 Motor de Datasets y Números de Serie Automáticos (`modules/records.js`)

Las vistas operativas de Compras, Ventas, Inventarios, Contabilidad y Costos consumen `GET / PUT /api/records`. Al guardar un dataset, el backend abre una transacción SQL, bloquea la secuencia en `business_serial_sequences` mediante `SELECT ... FOR UPDATE` y asigna folios correlativos con formato `SER-<MOD>-<VIEW>-000001` sin colisiones concurrentes.

5.3 Gestión de Empleados Multipaís (`modules/employees.js`)

Valida documentos oficiales según el código de país de la empresa: en Honduras (`HN`) exige DNI de 13 dígitos, RTN de 14 dígitos e IHSS; en México (`MX`) exige CURP (18 caracteres), RFC (12-13 caracteres) y NSS (11 dígitos). Implementa baja lógica conservando historial.

5.4 Reportes Contables y Financieros (`modules/accounting.js`)

Genera en tiempo real 6 reportes vía `GET /api/accounting/reports`: Estado de Resultados (`income-statement`), Balance General (`balance-sheet`), Balanza de Comprobación (`trial-balance`), Catálogo de Nómina (`payroll-catalog`), Utilidad por Línea de Venta (`sales-profit`) y Facturas de Compra (`purchase-invoice`).

6. Convenciones de Código y Reglas del Proyecto

- **Placeholders SQL con `?`:** Todas las sentencias SQL deben escribirse usando `?`. La función `postgresSql()` en `config/connection.js` los transforma automáticamente a `$1, $2, ...` antes de enviarlos a PostgreSQL.
- **Aislamiento Multi-tenant Obligatorio:** Toda consulta debe incluir filtro por `company_id`. Solo los usuarios con `isGlobalAdmin = true` pueden consultar o mutar datos de otra empresa mediante parámetro explícito.
- **Manejo Uniforme de Errores:** Arrojar instancias de `Error` adjuntando `error.statusCode` (400, 401, 403, 404, 429). El despachador central de `server.js` captura y responde en JSON.
- **Prevención de XSS en Cliente:** Escapar siempre cadenas dinámicas con `escapeHtml()` antes de insertarlas con `innerHTML`.

7. Estado Actual y Pendientes (Roadmap)

7.1 Módulos Concluidos y Funcionando

- **[COMPLETADO]** Autenticación propia, sesiones Bearer SHA-256, aprobación de usuarios y recuperación de contraseña.
- **[COMPLETADO]** Soporte multiempresa operativo para Vicmar (HN), Telas Olímpica (MX) y Grupo Cirsa (HN).
- **[COMPLETADO]** Directorio y CRUD de empleados con validación documental y fiscal por país.
- **[COMPLETADO]** Almacén de datasets dinámicos con asignación transaccional de números de serie.
- **[COMPLETADO]** Generación de reportes contables, estados financieros y nómina.
- **[COMPLETADO]** Integración con Supabase PostgreSQL y fallback local con `pg-mem`.

7.2 En Desarrollo / Próximas Funcionalidades (WIP)

- **[PENDIENTE]** Afectación automática de stock de inventario al cambiar el estado de recepciones de compra o pedidos de venta.
- **[PENDIENTE]** Integración de proveedor SMTP productivo en `config/mailer.js` (SendGrid / Resend / SES).
- **[PENDIENTE]** Emisión de comprobantes fiscales electrónicos (CAI SAR en Honduras y CFDI 4.0 en México).

7.3 Deuda Técnica

- **Migración de JSON a Tablas Relacionales:** Desacoplar progresivamente las vistas almacenadas en `business_datasets.rows_json` hacia tablas relacionales puras para soportar filtros SQL avanzados y paginación en servidor.
- **Rate Limiting Distribuido:** El limitador de peticiones en `server.js` reside en memoria del proceso (`Map`); para despliegues multi-instancia se recomienda migrarlo a PostgreSQL o Redis.